

KERNEL PANIC

Final Game Design Document — Prototype Revision (v2)

Dylan Overby · Multi-Agent AI for Game Development · Assignment #2 · Revised 25 July 2026 (final draft submitted 23 July 2026)

Turn-based cyberpunk roguelike · 2D desktop-UI presentation (“KP/OS”) · Playable prototype: **kernel-panic.higgsfield.app**

Revision notes: since the final draft, the complete game was built and shipped as a browser-playable prototype — tutorial, nine-day arc, finale, story, sound, and save system, all live. Every section below now describes what shipped and was measured, not what was planned. Changes from the final draft are flagged inline. The AI Architecture and Technical Strategy sections are rewritten from build experience: the agent pipeline described there is no longer a proposal — each loop in it was executed by hand at least once during the build, and the section records what those loops actually caught.

REVISED IN PROTOTYPE Sections carrying this marker explain what changed from the final draft and what playtesting or simulation data drove the change.

Executive Summary

Kernel Panic is a turn-based cyberpunk roguelike about inheriting your father's computer repair business and the machine in the back room he never let you touch. Your sister Rhea runs the front counter and insists the locked machine holds nothing but a nasty virus; you're certain it's more. Every run opens the same way: you dive it, you're not ready, it seals itself shut — and then you get to work, diving customers' infected devices to duel the intrusions inside. Each in-game day serves three tickets; diagnostics on every job let you plan your order before committing resources.

Every duel is a race to connect a signal to the core of a shared grid of scrambled pipe junctions — the same connect-the-pipes fantasy as Watch Dogs' hacking minigame, turn-based and head-to-head. The whole board is dealt face-up and scrambled; rotating junctions is the only movement verb. Your signal floods live: the moment a chain of junctions lines up, it claims the whole chain at once, and big cascades bank bonus RAM. Alongside routing, every diver carries the same three one-RAM programs — SCAN, ATTACK, DEFEND — which upgrade in tier and reconfigure into modes over a run. Winning is primarily a test of pathing efficiency; programs disrupt the opponent's route or protect yours, but never replace reading the grid well.

Win condition: reach the fixed finale day strong enough to win the duel against your father's machine and finally see what's inside — the AI companion he built for his son, sealed away until the son could stand on his own.

Loss condition: one rule, everywhere: Neural Strain reaching zero — whether zeroed instantly by a lost duel or bled dry by messy wins — ends the run. Each failed run advances the story.

Game Mechanics

Core Loop

REVISED IN PROTOTYPE The final draft's Analyze → Adjust Build → Dive chain shipped, then playtesting cut the mandatory Build stop: kit configuration is now an optional floating window, reachable from the ticket or the desktop, with its own dive button. The draft's random ability unlock became a curated augment draft (below).

A run spans a scripted, unwinnable tutorial dive (first run only), nine working days of three tickets each, and the Day 10 finale. Each ticket runs: **Analyze** (the diagnostic names the intrusion's dominant behavior — “Diagnostic flags halt traps. One wrong claim and you lose a whole turn. Scan early.”) → **Dive**, with kit configuration available but never forced. Clearing a job pays credits (40 + 25×tier, halved on a timeout win) and opens an **augment draft**: pick one of three augments never yet owned. Closing a day grants a choice — **+1 RAM per turn** (5 base, capped at 9) **or +1 tier on a program of your choice** — plus an optional **Night Patch** (60 credits for +12 Neural Strain). Strain starts at 100; messy wins chip it; a lost duel zeroes it, ending the run.

The Duel

REVISED IN PROTOTYPE The final draft still had each side placing nodes from a draw bag. Prototype playtesting replaced placement entirely: the full grid is pre-dealt scrambled, rotation is the only verb, and claiming happens by live signal flood. Every draft mechanic that survived (traps, redirects, telegraphs) now operates on this board.

The board is dealt corner-heavy (40% straights, 45% corners, rare tees and crosses) with dead slag cells for texture. You rotate junctions for 1 RAM per quarter turn — your own territory, or any open junction within **reach 2** of it. Both signals flood continuously: when rotation completes a connected chain back to a port, the whole chain claims instantly and permanently. A cascade of four or more junctions **banks** +1 RAM per four claimed (capped at 2) into your next turn. First flood to touch the core wins. Claimed territory is impassable to the enemy signal, so routing is also area denial — and if the machine ever walls you off completely, the duel ends immediately (**SEVERED**) rather than playing out a dead position, because severed routes can never heal. A 25-round cap resolves stalls toward whichever side sits closer to the core, ties to the player at extra strain cost.

The opponent is scripted logic, not a live agent: it commits to a per-turn rotation plan, casts at most one program per turn, and telegraphs every cast one beat ahead (a crimson cursor marks the aim, the strike lands next step). Its dominant mode — the one Analyze reported — is guaranteed to appear early, so the diagnostic is always honest.

Resources

REVISED IN PROTOTYPE Neural Capacity is cut. With the kit fixed at three always-carried programs there is nothing left for a loadout cap to govern; the draft's three shared resources are now two plus credits.

Resource	Role	Gained	Spent
----------	------	--------	-------

Neural Strain	The run's life bar	Restored to 100 at run start; Night Patch buys +12 for 60 credits	Chipped by messy wins — up to 40, scaling with how far the intrusion got, traps you triggered, and timeout wins; zeroed instantly by a lost duel. Zero, by any means, ends the run
RAM	Turn-by-turn action currency	5 per turn base, +1 per end-of-day pick (cap 9); up to 2 unspent carries over; cascades bank bonus RAM into the next turn	Rotations (1 per quarter turn) and programs (1 each) compete for the same pool
Credits	Run-scoped money	Job pay (40 + 25×tier, halved on timeout wins); 25-credit salvage when the augment pool runs dry	Night Patch (60 for +12 strain)

The Kit: Three Programs

REVISED IN PROTOTYPE *The largest post-draft change. The draft's eight verbs with ~24 tier-gated variants, a random unlock per win, consumable copies, and purchasable restocks played as advertised — and felt incoherent: too many verbs, no identity, unlock luck determining builds. The pool collapsed into three permanent programs with modes; testers could finally narrate their turns.*

Every diver always carries exactly three programs. Each costs **1 RAM**, always, and casts **once per turn**. Programs never unlock and are never consumed; they **upgrade** (tier, chosen at day close) and **reconfigure** (modes, unlocked by config augments and switchable freely between dives).

Program	Modes	What it does	Tier scaling
SCAN	—	Permanently reveals every armed node near your territory, whatever the trap type. Always 1 RAM, at every tier.	Reveal range 3 → 6 → whole board
ATTACK	REDIRECT (default) · ARM: HALT · ARM: SIPHON	Redirect twists an enemy or open junction a quarter turn, cutting power downstream. Halt traps cost the victim a full turn when sprung; siphon traps drain 2 RAM from their next turn into yours.	Targets per cast 1 → 2 → 3
DEFEND	PURGE (default) · LOCK · WARD	Purge disarms revealed traps (scan first — you can't defuse what you can't see). Lock freezes junctions against rotation and redirects for 2 rounds. Ward blankets an area against new traps for 2 rounds.	Targets per cast 1 → 2 → 3; ward radius 1 → 2 → 3

Every mode has a counter that the opponent also plays: scan finds arms, purge clears them, ward pre-empts them, lock stops redirects. The machine's kit is built from the same table — its tier and mode spread widen with the ticket's threat tier.

Augments

Fourteen augments, drafted one per cleared job from a three-card offer. The four **config drivers** unlock modes (halt, siphon, lock, ward) and are guaranteed a draft slot while any remain unowned; the ten **boosts** bend the economy around a build. Owned augments never reappear.

Augment	Kind	Effect
HALT DRIVER / SIPHON DRIVER	Config	Unlock ATTACK's halt / siphon trap modes
CLAMP DRIVER / WARD DRIVER	Config	Unlock DEFEND's lock / ward modes
LONG ARMS	Boost	Rotate open junctions up to 3 steps out instead of 2 — bigger setups, bigger cascades
SURGE CACHE	Boost	Every RAM-banking cascade banks +1 extra
DEEP SIPHON / TRIPWIRE	Boost	Siphon traps steal +1 RAM / halt traps also burn 2 RAM off the victim's next active turn
CHEAP SHOT / HOT BOOT	Boost	First ATTACK each dive is free / start every dive with +2 RAM
BULWARK / TAP LINE	Boost	DEFEND also freezes its targets 1 round / SCAN also traces the intrusion's planned route
CARRY CACHE / ECHO TAP	Boost	Carry 4 unspent RAM instead of 2 / each of your traps that fires banks +2 RAM

Cascades

REVISED IN PROTOTYPE *Simulation data forced one refinement: cascade RAM paid out immediately compounded into a route-finished-in-one-turn snowball (average duel length collapsed to 1.5 rounds in a 200-seed sweep). Banked into the following turn, the same reward is pure tempo without the degenerate spiral.*

Cascades are the game's signature satisfying moment — a long scrambled chain snapping into place and claiming six junctions at once — so they carry real utility: banked RAM, deterministic, never a coin flip between reward types. Reach-2 rotation exists substantially to serve them: being able to pre-align junctions two steps ahead of your border is what makes engineered multi-junction cascades a plannable play rather than a lucky one.

Structure, Presentation & Finale

The game presents as **KP/OS**, a retro-futuristic pixel desktop: BIOS boot, a login screen with three deletable user slots (one save per user), desktop icons for JOBS / LOADOUT / MANUAL / LEDGER, draggable windows, and a taskbar clock showing the in-game day. The dive takes the full screen. A mid-dive refresh is the designed safe abort: the run resumes at the day board with the ticket still open. The father's journal, **DAD.LOG**, unlocks entry by entry across runs — the will, the quarantine story, Meridian Neurocare receipts dated four days before he died — keeping the mystery of the back-room machine in view between dives.

Tutorial

REVISED IN PROTOTYPE *The draft's "one scripted tier-0 config, no bespoke tutorial system" failed its first real playtest: the machine won before the player touched a program. The shipped tutorial stage-manages the kit — programs boot OFFLINE and unlock as the lesson demands them (the machine plants a trap → SCAN unlocks; you scan it → DEFEND; you purge it → ATTACK) — while the machine is throttled to a fraction of its RAM. Unwinnability became structural rather than tuned: the moment your flood touches the tutorial core, every port slams shut. No generator margin math, no way to break it.*

The player learns all three programs by using each one for real, gets one full-kit turn to feel the complete loop, and then the machine stops pretending, plays at full speed, and seals. Neural Strain zeroes; day one begins. Losing is still the tutorial's final lesson.

Finale & Story

The finale falls on Day 10, every run, ready or not: the biggest grid (13×11), 10 RAM per turn, every mode at full width, already four nodes deep when you sit down. Mechanically it is an ordinary duel; that is the point. Winning opens the machine — Patch, the companion the father built for his son, named for the thing that holds a broken thing together while it mends. Failed runs advance the story: Rhea's virus theory erodes run by run, and a father fragment surfaces after each attempt. Losing is how the player learns who their father was.

Difficulty (measured)

The shipped escalation curve, with win rates from the 200-seed-per-day simulation harness (a kit-less proxy player — a human with augments runs hotter):

Day	Grid	Machine RAM	Head start	Ticket tiers	Proxy win rate
1	9×7	6	0	1 / 1 / 1	82%
2	9×7	6	0	1 / 1 / 2	77%
3	9×9	6	1	1 / 2 / 2	74%
4	9×9	6	2	2 / 2 / 3	56%
5	11×9	7	2	2 / 3 / 3	58%
6	11×9	7	2	3 / 3 / 3	56%

7	11x9	7	3	3 / 3 / 4	49%
8	11x11	8	3	4 / 4 / 4	42%
9	11x11	9	4	4 / 4 / 5	39%
10	Finale: 13x11, 10 RAM, head start 4, full kit				25%

The tutorial config must post 0 wins in 200 seeds before any deploy; the harness enforces it.

AI Architecture

REVISED IN PROTOTYPE *Rewritten from experience. The final draft proposed six agents; building the prototype exposed two missing specialists (abilities and UX), showed that dialogue is a pacing job rather than a string-generation job, and reframed “Development” from a line item into the orchestrating agent everything else reports through. Every loop below was executed at least once, by hand, during the build; the section records what each one caught.*

The principle is unchanged: every agent runs during development, not during play. Each produces content in a defined, structured format — JSON profiles, config tables, tagged copy, assets, pass/fail reports — baked into a static library the shipped game draws from. The game makes no API calls while it is being played. Prose handoffs between agents are banned: structure is the contract, and the Orchestrator is the only agent that touches the codebase.

Orchestrator — Claude Fable 5. Formerly the “Development (build)” row. Reframed as the integrating agent: it consumes every other agent's artifacts, writes and maintains all code, keeps the engine deterministic, and runs the verification gates (typecheck, duel simulation, run-layer invariants, production build, live smoke test) before anything ships. It also arbitrates when two agents' outputs collide — a new augment that breaks a day's win-rate target, a story still that contradicts the bible. It runs on Fable 5 because integration is where all contexts meet at once: engine, balance data, canon, and UI state have to be held simultaneously, and it is the only agent whose mistakes compound.

Loremaster — canon authority. Expanded: alongside the setting bible, it now keeps the **story ledger** — the ground truth of the underlying story. What actually happened to Dad, what Patch is, what Rhea believes versus what she knows, what the player can possibly know after run N. Every outward-facing artifact — customer profiles, dialogue, journal entries, story art — passes its approve/revise gate. One agent owning truth is what keeps eleven runs of drip-fed mystery from contradicting itself.

Narrative Director — story pacing. Formerly “Outcome & Dialogue,” substantially buffed. The encounter generator already covers duel-outcome flavor; this agent's job is the **reveal schedule**: how the mystery of Dad's computer unfolds, how the opening context is set, how the running conversation with Rhea erodes from “it's just a nasty virus” toward the truth. Its second responsibility is environmental hinting — the artifacts that keep the ultimate goal in the player's peripheral vision. The will and the hospital receipts surfacing in DAD.LOG are the template: diegetic, optional, cumulative. It drafts within the Loremaster's ledger and commissions the Art Lead for key-moment stills.

Customer/Encounter Generator — the “One Wow.” Unchanged core: one structured JSON profile per job — customer name, device, flavor text, threat tier, dominant mode — feeding both the diagnostic readout and the duel's starting conditions, so the player meets a specific character with a specific,

telegraphed opponent on every ticket. It now also consumes the Ability Agent's current catalog, so a customer's dominant mode always names a real, current mechanic and the Analyze tell stays honest.

Ability Agent — new. Owns the ability catalog: attack and defend modes, scan scaling, and the augment pool. Its design brief is explicitly **synergy and counterplay** — every ability ships as part of a pair or triangle (halt traps ↔ scan/purge, redirect ↔ lock, arming ↔ ward) rather than as an isolated effect. It sits inside the balance loop: encounter generation assigns its abilities to opponents, the Arc Composer schedules them across days, Validation measures the result, and the Ability Agent adds, retunes, or retires entries in response. The prototype's kit is this loop's output run by hand: cascade banking, reach-2 rotation, and the config/boost augment split all came out of simulation findings, and the entire draft-era ability pool was retired by it.

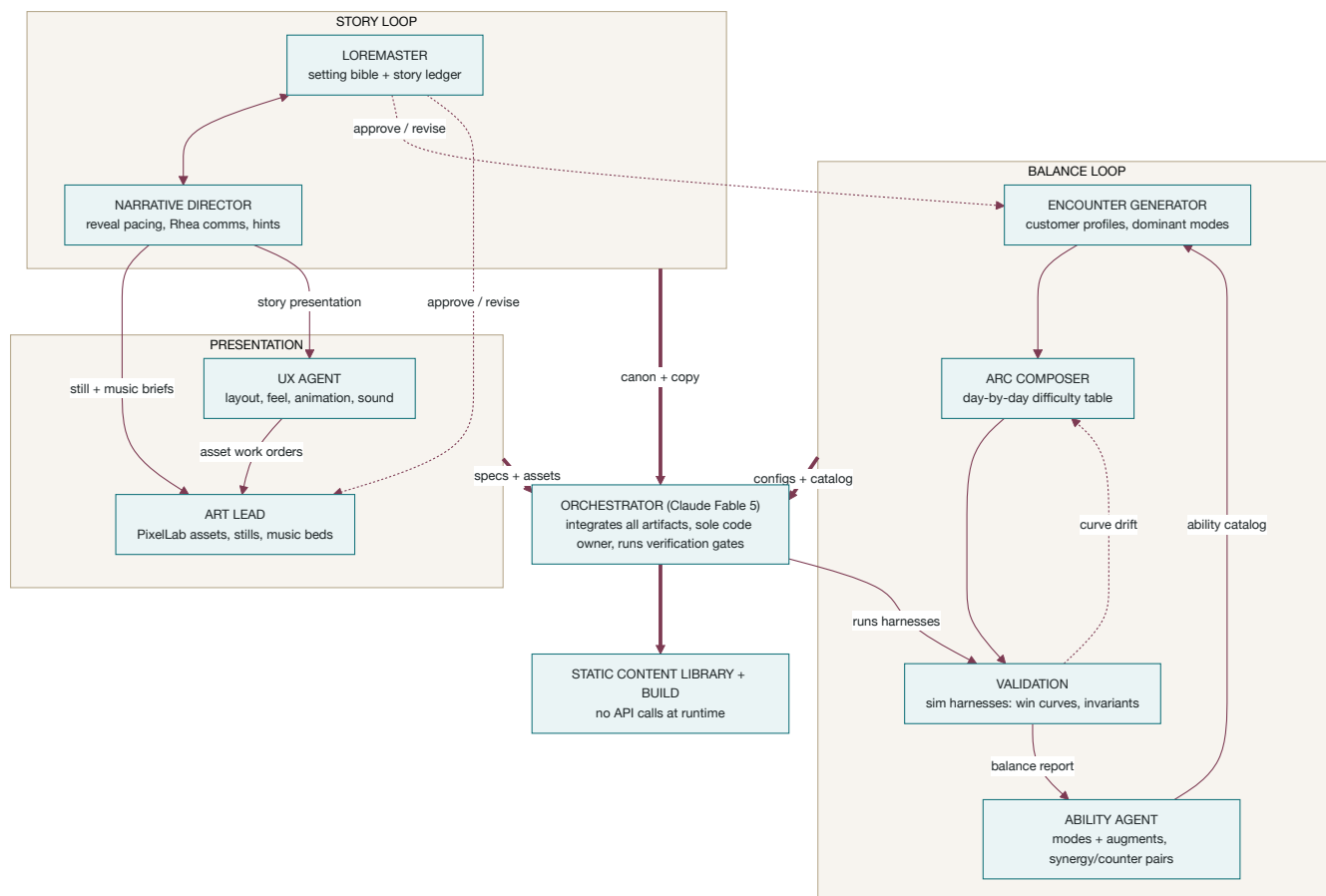
Arc Composer. Unchanged role, proven surface: the entire difficulty curve is one table — grid, machine RAM, aggression, cast frequency, route-cost target, head start, ticket tiers, per day. Every balance change of the build touched only this table, which is what made nightly retuning cheap.

Validation. Now concrete rather than aspirational: it operates the two harnesses that exist in the repo — a 200-seed-per-day duel simulator reporting win/round/strain-chip distributions, and a full run-layer simulator asserting state-machine invariants across complete runs. Its report is distributions against targets, not vibes. Build evidence for why it gates everything: it caught the cascade snowball (duels collapsing to 1.5 rounds), the silently degenerate draft tutorial, and the 2% of boards where the machine's head start walled the player off before round one.

UX Agent — new. This game *is* its interface — a desktop OS the player lives in — so presentation gets its own specialist rather than falling to the art or build line. It owns the layout and information hierarchy of KP/OS (windows, dock, targeting flows), interaction feel (telegraph legibility, animation timing, screen-shake tiers, cascade pops), and **sound design**: every cast, claim, trap, and cascade has a synthesized voice, and the audio palette is authored in code as part of the interaction, not attached afterward. It writes work orders to the Art Lead for the pixel assets it needs (chrome, icons, cursors) and collaborates with the Narrative Director on how story moments present (typed dialogue pacing, still framing, music cues).

Art Lead. Revised pipeline: purchased icon/UI packs are cut entirely. All pixel art is generated with **PixelLab**, with the game's fixed palette hexes pinned in every prompt and a deterministic post-pass (nearest-neighbor downscale plus palette quantize) that keeps output on-model across regenerations; larger canvases (wallpaper, story stills) and music beds fall to Higgsfield generation under the same palette rules. It is deliberately a fabrication shop: it fulfills work orders from the UX Agent and Narrative Director rather than self-directing, and its outward-facing output passes the Loremaster's gate.

Where sound lives. Sound design sits under the UX Agent, not a separate sound agent. In this game audio is interaction feedback — the cascade arpeggio that scales with chain length, the two-beat telegraph sting, the trap crack — and it is tuned in the same pass as the animation it accompanies. A standalone sound agent would put an organizational boundary through the tightest feedback loop in the project. The build bears this out: the prototype's worst audio bug (an entire inaudible palette) was found and fixed with UX-style instrumentation of the running interface, not with better composition. The one carve-out is music: longer beds are asset production, so the UX Agent writes the brief (mood, tempo, scene) and the generation pipeline fulfills it like any other art order.



The agent org. Solid heavy arrows are artifact flows into the Orchestrator; dashed arrows are review gates and feedback; the Balance Loop cycles until Validation's distributions hit the arc targets.

Pipeline order. The balance loop and story loop run in parallel; the Loremaster gates everything outward-facing from either. The UX Agent and Art Lead iterate continuously against whatever mechanics and story beats are current. The Orchestrator integrates on its own cadence, and nothing enters the shipped library until Validation's harnesses pass — including the standing rule that the tutorial posts zero wins in 200 seeds.

Technical Strategy

REVISED IN PROTOTYPE Two revisions. Unity is cut: the prototype shipped as a TypeScript web app and proved the presentation needs nothing heavier — the entire game is UI, and the browser is the native home of a fake desktop OS. Purchased asset packs are cut: all art is generated (PixelLab primary), which the palette-pinning pipeline made reliable enough to bet on.

Kernel Panic is a TypeScript web application: the whole game is pure reducer functions over a seeded RNG, rendered as a 2D desktop UI, deployed to the same URL every build. Determinism is the load-bearing decision, made for the agent pipeline rather than the player: the same seed always produces the same board and the same opponent decisions, which is what lets Validation regress balance numerically, lets a bug report from a live player be reproduced exactly from their seed, and lets the opponent telegraph moves without random drift between simulation and play. All balance lives in one table (the Arc Composer's surface); all kit numbers live in one content module (the Ability Agent's surface). The verification gate that

ran before every deploy: typecheck, 200-seed duel sims for all ten day-configs plus tutorial, full-run invariant sims, production build, and a live smoke test.

Art: PixelLab generation with palette hexes pinned in every prompt, then a deterministic downscale-and-quantize pass; Higgsfield image models for large canvases and synthesized music beds. Sound effects are code: a compact synthesizer with a preset palette, authored and tuned by the UX pass. Figures below count input and output tokens together, since both draw against plan usage.

Agent / task	Model	Calls	Tokens per call (in + out)	Estimated total
Orchestrator (integrate + build)	Fable 5	~60 turns	~8,000 average per turn	~480,000
Customer/Encounter Generator	Sonnet 5	60	~1,200 + ~1,500 = 2,700	162,000
Ability Agent	Sonnet 5	40	~1,100 + ~900 = 2,000	80,000
Narrative Director	Sonnet 5	50	~700 + ~500 = 1,200	60,000
Loremaster (reviews + ledger)	Sonnet 5	100	~900 + ~250 = 1,150	115,000
Arc Composer	Sonnet 5	12	~800 + ~500 = 1,300	15,600
UX Agent	Sonnet 5	60	~900 + ~600 = 1,500	90,000
Validation (harness runs + reports)	Haiku 4.5	240	~300 + ~400 = 700	168,000
Art Lead (prompt authoring)	Haiku 4.5	120	~250 + ~150 = 400	48,000
Total across all models				≈1,218,600 tokens

The Orchestrator row is the least predictable line — long coding sessions resend conversation history every turn — and is controlled by clearing context between subtasks rather than by call count. Art Lead's real budget is PixelLab generation credits, not tokens; prompt authoring is deliberately routed to Haiku.

Constraints & Scope

- **One person, remaining weeks.** The prototype exists and is live, which retires the engine risk entirely; the remaining budget goes to content breadth (more customers, more journal, more augments), which is exactly what the agent pipeline is shaped to produce.
- **Generation credits are capped.** PixelLab jobs are the scarce resource, not tokens. Palette pinning plus the deterministic post-pass means a regeneration is cheap to accept or reject, so the budget buys attempts, not obligations.
- **Context windows are fixed.** The Loremaster reviews against a condensed bible and ledger summary, never the full generation history; the Orchestrator clears context between subtasks — the single most effective cost control observed during the build.
- **Validation loops must terminate.** Generate-and-check caps at three retries before a failing config is flagged for manual review. Ability churn is bounded the same way: catalog changes enter only through the balance loop, each carrying before/after win curves, so the Ability Agent cannot thrash the meta.
- **Model routing follows constraint, not prestige.** Haiku carries the highest-call-count lines (Validation, art prompts) to keep load off the Sonnet cap; Fable 5 is reserved for the one seat where cross-domain context cannot be split — the Orchestrator.